DATABASE SYSTEMS

Relational Algebra Lab

Student Tutorial and Practical Guide

What you need: A web browser, the file lab_dataset.txt, and a place to type your answers. You do not need to install a database, Python, or a VS Code extension.

Lab length: about 30 minutes | Main work: 10 practical questions + one short discussion

Use this guide in order. The worked examples are for practice. They do not give away the answers to the required questions.

1. Quick start

Today you will write and run relational algebra expressions in RelaX. The main idea is simple: each operator takes one or more relations and produces another relation.

RA in one sentence: SQL usually states the result you want. Relational algebra shows the operations used to build that result.

The five-step routine

Read the output → Choose relations → Build inside out → Run and inspect → Submit clearly

Before you begin

[ ] I have lab_dataset.txt.

[ ] I have opened this guide and a document for my answers.

[ ] I know where I will save my final work.

[ ] I will keep the instructor answer key closed during the lab.

A few rules that save time

- Read nested expressions from the inside outward.
- Use explicit join conditions unless you have checked every common attribute name.
- Before union, intersection, or difference, make sure both inputs have the same columns and compatible types.
- Relations are unordered. A different row order is not a different answer.
- Do not submit screenshots only. Submit the expression as text.

2. Open RelaX and load the dataset

Open: https://dbis-uibk.github.io/relax/

1. Choose Create new Dataset or open the Group Editor. The wording may be slightly different depending on the screen size.
2. Open lab_dataset.txt and copy the whole file.
3. Paste the text into the dataset editor.
4. Save or load the group named Campus Collaboration Lab.
5. Open the Relational Algebra editor.
6. Type Student and press Ctrl+Enter (or Ctrl+Return).
7. Check that the Student relation appears with 13 tuples.

Setup check: If Student runs, the dataset is ready. Do not continue if the editor says that Student is unknown. Go back and make sure the complete dataset text was pasted and loaded.

Expected relation sizes

Relation	Tuples	Relation	Tuples
Student	13	Course	8
Enrollment	30	Club	5
Membership	20	Project	8
WorksOn	20		

If the website does not run

- Refresh the page once and reload the dataset.
- Try a private/incognito window or a different browser.
- Check that quotes are straight quotes, for example 'C101', not curved quotation marks.
- Use the supplied lab_dataset.txt instead of retyping the tables.

3. Know the dataset

You do not need to memorize every row. You do need to know which relation contains each attribute and how the relations connect.

Relation	Attributes	What it stores
Student	sid, sname, dept, year	One row per student
Course	cid, title, dept, level	One row per course
Enrollment	sid, cid, term, grade	Students taking courses
Club	club_id, club_name, category	One row per club
Membership	sid, club_id, role	Students joining clubs
Project	pid, ptitle, theme, lead_sid	Projects and their leads
WorksOn	sid, pid, hours	Students working on projects

Key connections

```
Student.sid  --> Enrollment.sid
Course.cid   --> Enrollment.cid
Student.sid  --> Membership.sid
Club.club_id --> Membership.club_id
Student.sid  --> WorksOn.sid
Project.pid  --> WorksOn.pid
Student.sid  --> Project.lead_sid
```

Important data details: Two students are named Alex Lee. Some students have no club. One student has no enrollment. Student and Course both contain an attribute named dept. These details are intentional and matter in several questions.

4. RelaX syntax you will use

You may type the plain-text form. RelaX can also display the mathematical symbols.

Idea	Math	RelaX text	Use it for
Selection	σ	sigma	Keep rows that meet a condition
Projection	π	pi	Keep selected columns
Rename	ρ	rho	Give a relation or attribute a new name
Union	∪	union	Rows in either compatible relation
Intersection	∩	intersect	Rows in both compatible relations
Difference	−	-	Rows in the first relation but not the second
Cross product	×	cross join	Pair every row from two relations
Join	⋈	join	Combine related rows

Small syntax examples

Selection: keep DATA students

```
sigma dept = 'DATA' (Student)
```

Projection: keep only student ID and name

```
pi sid, sname (Student)
```

Rename and join: give each input a short alias

```
rho S (Student) join S.sid = E.sid rho E (Enrollment)
```

Set difference: course IDs that never appear in Enrollment

```
pi cid (Course) - pi cid (Enrollment)
```

Read from the inside out

```
pi sid, sname (
  sigma dept = 'DATA' and year >= 3 (Student)
)
```

First select DATA students in year 3 or 4. Then keep only sid and sname.

Selection and projection are easy to mix up: RA selection (sigma) keeps rows. RA projection (pi) keeps columns. In SQL, WHERE behaves like RA selection, while the SELECT list behaves like RA projection.

5. Worked examples

Run these examples before starting the marked questions. They show the usual pattern: filter, join if needed, then project the final columns.

Example A: selection followed by projection

Task: return the IDs and names of students in the DATA department.

Mathematical RA: π_sid,sname(σ_dept='DATA'(Student))

RelaX:

```
pi sid, sname (
  sigma dept = 'DATA' (Student)
)
```

Check: the result should contain S03 Cara Singh, S05 Erin Park, and S10 Jae Patel.

Example B: join, selection, and projection

Task: show each student who works more than 8 hours on a project. Return sid, sname, pid, and hours.

```
pi S.sid, S.sname, W.pid, W.hours (
  sigma W.hours > 8 (
    rho S (Student) join S.sid = W.sid rho W (WorksOn)
  )
)
```

Check: the output has seven tuples. Click the join and selection nodes in the operator tree to see how the relation becomes smaller.

Example C: a “not exists” pattern

Task: return course IDs that have no enrollment.

```
pi cid (Course)
-
pi cid (Enrollment)
```

Check: the result is C108. The first relation is the full set of course IDs. The second is the set of course IDs that appear in Enrollment. Subtracting the second from the first leaves courses with no matching enrollment.

Patterns to remember

Words in the question	Useful RA pattern
and	One selection predicate with and, or an intersection when comparing two derived sets
or (or both)	Union of two compatible result relations
both	Intersection or a self-join
not / no related row	Start with the valid universe, then subtract the matched set
at least two different	Use two renamed copies and require different item IDs
every / all	Eligible set minus rows with a counterexample

Natural join warning: Natural join matches every attribute name shared by the two inputs. It does not know the intended foreign keys. In this dataset, Student and Course both have dept, so an unplanned natural join can remove valid enrollments.

6. The 30-minute lab path

Complete the questions in this order. For every required question, submit the mathematical RA expression, the executable RelaX expression, and the result relation.

Time	Work
0–4 min	Load the dataset and run one worked example
4–8 min	Q1 and Q3
8–12 min	Q4 and Q6
12–16 min	Q7 and Q9
16–20 min	Q11 and Q12
20–25 min	Q14 and Q20
25–28 min	Q22 pair discussion
28–30 min	Final check and submission

Required exercises

Q1. Database distinction

Return the student ID, student name, and grade for every student who earned at least 80 in C101 (Database Systems).

Hint: Filter Enrollment first. Then join it to Student on sid.

Q3. The two Alex Lees

Return each Alex Lee's ID, name, course ID, and course title for every course in which either Alex is enrolled.

Hint: A name is not a key. Keep sid and use three relations.

Q4. AI or Robotics Club

Return the IDs and names of students who belong to AI Society or Robotics Club, or both. Your solution must use union.

Hint: Build two result relations with exactly the same schema before applying union.

Q6. Students without a club

Return the IDs and names of students who have no Membership tuple.

Hint: Start with all students and subtract the students who appear in a Student-Membership join.

Q7. Cross-department study

Return sid, sname, cid, and title for every enrollment where the student's department differs from the course's department.

Hint: Use explicit key joins, then compare the two dept attributes.

Q9. Where did the second Alex go?

Evaluate π_sname(Student). State the number of tuples and explain why ordinary SQL SELECT sname FROM Student behaves differently.

Hint: Projection uses set semantics, so identical projected tuples are removed.

Q11. CS students missing C101

Return the IDs and names of Computer Science students who are not enrolled in C101.

Hint: Create the set of CS students and subtract the CS students found in C101.

Q12. The C101/C102 core

Return the IDs and names of students enrolled in both C101 and C102.

Hint: Find the student-ID set for each course. Then use intersection or a self-join.

Q14. Two high marks

Return the IDs and names of students with grades of at least 85 in at least two different courses.

Hint: Use two renamed copies of Enrollment. Match the same sid and require different cid values.

Q20. No grade below 80

Return students who have at least one enrollment and whose every grade is at least 80. Do not include students with no enrollment.

Hint: Start with enrolled students. Subtract students who have a grade below 80.

Pair discussion: Q22 - push the filter down

Compare the two expressions below. Decide whether they are equivalent for every valid database instance. Then state which plan normally processes fewer tuples before the join.

```
A: pi S.sid, S.sname, E.grade (
    sigma E.cid = 'C101' and E.grade >= 80 (
      rho S (Student) join S.sid = E.sid rho E (Enrollment)
    )
  )

B: pi S.sid, S.sname, E.grade (
    rho S (Student) join S.sid = E.sid
      rho E (sigma cid = 'C101' and grade >= 80 (Enrollment))
  )
```

Write two or three sentences. Do not answer only “A” or “B.”

Early-finisher challenge: Q28

For student S05, compare Student join Enrollment join Course with explicit theta joins on sid and cid. Find the rows lost by the natural join and write a safe rewrite using renaming.

7. How to write and submit each answer

Use the same format for every practical question. Clear work is easier to debug and easier to mark.

Answer template

Q__ Title

1. Mathematical RA:

   _____

2. RelaX expression:

   _____

3. Output schema and tuples:

   _____

4. Short explanation, when requested:

   _____

Final checklist

[ ] Every required question has a mathematical RA expression.

[ ] Every required question has a runnable RelaX expression.

[ ] Every result includes attribute names.

[ ] Set-operation inputs have compatible schemas.

[ ] Self-joins use different aliases.

[ ] Q9 includes the RA-versus-SQL explanation.

[ ] Q22 includes both equivalence and optimization reasoning.

[ ] I saved the file with my name or student ID.

Practical lab marking

The practical lab is marked out of 40 points. Your instructor may adjust the weighting, but this is the default rubric.

Area	Points	What earns full credit
Correct query results	20	Correct relation for the 10 required questions
Legal schemas and operators	6	Valid attributes, joins, aliases, and compatible set inputs
RelaX execution	4	Expressions run with the supplied dataset
Reasoning	5	Clear explanations for the set, negative, and universal questions
Q22 discussion	3	Correct equivalence and smaller-intermediate explanation
Presentation	2	Readable, labeled work with output schemas

Separate multiple-choice quiz

If your instructor assigns the Excel quiz, open ra_mcq_student_quiz.xlsx. Enter A, B, C, or D in the Student Answer column or on the Answer Sheet. Save a copy with your name. If all 30 questions are assigned, the maximum raw score is 88 points. The practical lab and the Excel quiz are separate activities.

8. Quick Q&A and troubleshooting

What does RA mean?

Relational Algebra. It is a formal way to describe operations on relations.

Do I need to download software?

No. RelaX runs in the browser. VS Code or Word is only for typing your submission.

Why does pi sometimes reduce the number of rows?

Pure relational algebra uses set semantics. If projection creates identical tuples, one copy remains.

Why do I need rho?

Use rho when the same relation appears more than once or when attribute names would be ambiguous.

Why does my union, intersection, or difference fail?

The two inputs probably have different numbers of attributes, different types, or incompatible attribute positions.

Why did natural join remove rows?

Natural join matched every common attribute name, possibly including a column you did not intend to use.

Does row order matter?

No. Relations are unordered. Check the attributes and the set of tuples.

My query runs but the result is wrong. What should I inspect?

Click the operator-tree nodes. Find the first intermediate relation that is not what you expected.

Does the correct result on this dataset prove two expressions are always equivalent?

No. Equivalence must hold for every valid database instance. One counterexample is enough to disprove it.

A simple hint ladder

1. Write the output attributes first.
2. List the relations that contain those attributes.
3. Write the join keys between those relations.
4. Add the filter condition.
5. Project the final columns last.
6. Run each inner part separately if the full query is hard to debug.

Before asking for the full answer: Show the expression you tried and the first intermediate result that looks wrong. A small correction is usually more useful than replacing the whole query.

One-page reference

Need	Pattern
Filter rows	sigma condition (R)
Choose columns	pi a, b (R)
Rename a relation	rho X (R)
Join on keys	rho A (R) join A.key = B.key rho B (S)
Either set	A union B
Both sets	A intersect B
In A but not B	A - B
All pairs	A cross join B

Checks before running

[ ] Are all attribute names available at the point where I use them?

[ ] Did I join on the correct key attributes?

[ ] Did I use aliases when a relation appears twice?

[ ] Are set-operation inputs compatible?

[ ] Did I project the requested output columns only?

[ ] Did I avoid an unintended natural join?

Useful translation

SQL idea	RA idea
WHERE condition	Selection (sigma)
SELECT column list	Projection (pi)
JOIN ... ON ...	Theta/equi-join
UNION	Union
INTERSECT	Intersection
EXCEPT	Set difference
DISTINCT	Built into pure RA set semantics

Work inside out. Check the schema at every step. Keep your final answer simple.

Based on Lecture 03: Relational Algebra and the Campus Collaboration lab materials.